Assignment 9.2 - Governance, Audit, and Controlled Orchestration

Part II – Guided AI-Orchestrated Build

Feature Selection

For the controlled AI-orchestration task, I selected the Featured Taco List Component. The component will display at least four featured tacos, including each taco's name, description, price, and a label such as "Popular," "Spicy," or "New." One taco will be identified and conditionally displayed as the Chef's Pick. The component will be implemented as a new standalone Angular component with its own internal data and will not modify any existing application components or business logic. It will be integrated through a new application route so that the feature remains isolated from the existing order workflow. Three unit tests will verify the featured taco list, the required taco details, and the Chef's Pick behavior.

Deepseek Prompt:

```
Create a single standalone Angular 22 component named
FeaturedTacosComponent for an existing Virtual Taco Stand application.
The component must be completely self-contained. Do not modify, rewrite,
or generate code for any existing component, service, model, stylesheet,
route configuration, or other application file.
Requirements:
Create featured-tacos.component.ts.
Use a standalone Angular component.
Use an inline HTML template.
Do not create a component-specific CSS file or inline styles.
Define an internal typed collection containing at least four featured
tacos.
Each taco must contain a name, short description, numeric price, and one
label such as Spicy, Popular, or New.
Use Angular template list rendering to display all featured tacos.
Identify one taco as the Chef's Pick and use conditional rendering to
display a clear Chef's Pick message for that taco only.
Format prices as currency using Angular-supported template functionality.
Keep all feature data and display logic inside this component.
Do not introduce services, HTTP requests, external libraries, application-
wide state, or changes to existing application architecture.
Also create featured-tacos.component.spec.ts containing exactly three
meaningful Jasmine/Karma unit tests:
verifies that at least four featured tacos are rendered;
```

verifies that rendered taco information includes the expected name, price, and featured label;
verifies that exactly one taco is identified as the Chef’s Pick.
The tests must use Angular TestBed and must test observable component behavior rather than internal implementation details.
Return only the contents of the two new files. Do not modify or suggest modifications to any existing application files.

Deepseek Response:

featured-tacos.component.ts

```
import { Component } from '@angular/core';
import { CurrencyPipe, NgFor, NgIf } from '@angular/common';

interface FeaturedTaco {
  name: string;
  description: string;
  price: number;
  label: 'Spicy' | 'Popular' | 'New' | 'Classic';
  isChefsPick: boolean;
}

@Component({
  selector: 'app-featured-tacos',
  standalone: true,
  imports: [NgFor, NgIf, CurrencyPipe],
  template: `
    <div class="featured-tacos-container">
      <h2>Featured Tacos</h2>
      <div class="taco-list">
        <div *ngFor="let taco of featuredTacos" class="taco-card">
          <div class="taco-header">
            <h3>{{ taco.name }}</h3>
            <span class="taco-label" [class.spicy]="taco.label === 'Spicy'"
                        [class.popular]="taco.label === 'Popular'"
                        [class.new]="taco.label === 'New'"
                        [class.classic]="taco.label === 'Classic'">
              {{ taco.label }}
            </span>
          </div>
          <p class="taco-description">{{ taco.description }}</p>
          <div class="taco-footer">
            <span class="taco-price">{{ taco.price | currency }}</span>
            <div *ngIf="taco.isChefsPick" class="chefs-pick">
              🏆 Chef's Pick!
            </div>
          </div>
        </div>
```

```
      </div>
    </div>
  `,
  styles: []
})
export class FeaturedTacosComponent {
  featuredTacos: FeaturedTaco[] = [
    {
      name: 'Carne Asada',
      description: 'Grilled steak with fresh cilantro and onions',
      price: 4.99,
      label: 'Popular',
      isChefsPick: false
    },
    {
      name: 'Ghost Pepper Carnitas',
      description: 'Slow-cooked pork with ghost peppers - not for the faint of heart',
      price: 5.49,
      label: 'Spicy',
      isChefsPick: true
    },
    {
      name: 'Baja Fish',
      description: 'Crispy beer-battered cod with cabbage slaw and lime crema',
      price: 6.25,
      label: 'New',
      isChefsPick: false
    },
    {
      name: 'Al Pastor',
      description: 'Marinated pork with pineapple and traditional spices',
      price: 4.75,
      label: 'Classic',
      isChefsPick: false
    },
    {
      name: 'Mushroom & Poblano',
      description: 'Roasted mushrooms and poblano peppers with queso fresco',
      price: 5.99,
      label: 'New',
      isChefsPick: false
    }
  ];
}
```

featured-tacos.component.spec.ts

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { FeaturedTacosComponent } from './featured-tacos.component';
import { CurrencyPipe } from '@angular/common';
```

```
describe('FeaturedTacosComponent', () => {
  let component: FeaturedTacosComponent;
  let fixture: ComponentFixture<FeaturedTacosComponent>;
  let compiled: HTMLElement;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [FeaturedTacosComponent]
    }).compileComponents();

    fixture = TestBed.createComponent(FeaturedTacosComponent);
    component = fixture.componentInstance;
    compiled = fixture.nativeElement;
    fixture.detectChanges();
  });

  it('should render at least four featured tacos', () => {
    const tacoCards = compiled.querySelectorAll('.taco-card');
    expect(tacoCards.length).toBeGreaterThanOrEqual(4);
    expect(component.featuredTacos.length).toBeGreaterThanOrEqual(4);
  });

  it('should render taco information including name, price, and featured label', () => {
    const tacoCards = compiled.querySelectorAll('.taco-card');

    tacoCards.forEach((card, index) => {
      const taco = component.featuredTacos[index];

      // Check name
      const nameElement = card.querySelector('h3');
      expect(nameElement?.textContent).toContain(taco.name);

      // Check price with currency formatting
      const priceElement = card.querySelector('.taco-price');
      const currencyPipe = new CurrencyPipe('en-US');
      const expectedPriceText = currencyPipe.transform(taco.price);
      expect(priceElement?.textContent).toContain(expectedPriceText);

      // Check label
      const labelElement = card.querySelector('.taco-label');
      expect(labelElement?.textContent).toContain(taco.label);
    });
  });

  it('should identify exactly one taco as the Chef\'s Pick', () => {
    const chefsPickElements = compiled.querySelectorAll('.chefs-pick');
    expect(chefsPickElements.length).toBe(1);

    // Verify the component data matches
    const chefsPickTacos = component.featuredTacos.filter(taco => taco.isChefsPick);
    expect(chefsPickTacos.length).toBe(1);
```

```
    // Verify the displayed Chef's Pick matches the data
    const chefsPickTaco = chefsPickTacos[0];
    const chefsPickElement = chefsPickElements[0];
    expect(chefsPickElement.textContent).toContain('Chef\'s Pick');

    // Find which taco card contains the Chef's Pick element
    const tacoCards = compiled.querySelectorAll('.taco-card');
    let foundChefsPick = false;

    tacoCards.forEach((card, index) => {
      const chefsPickInCard = card.querySelector('.chefs-pick');
      if (chefsPickInCard) {
        const nameElement = card.querySelector('h3');
        expect(nameElement?.textContent).toContain(chefsPickTaco.name);
        foundChefsPick = true;
      }
    });

    expect(foundChefsPick).toBeTrue();
  });
});
```

Revision of AI produced results

featured-tacos.component.ts

```
import { CurrencyPipe } from '@angular/common';
import { Component } from '@angular/core';

interface FeaturedTaco {
  name: string;
  description: string;
  price: number;
  label: 'Spicy' | 'Popular' | 'New' | 'Classic';
  isChefsPick: boolean;
}

@Component({
  selector: 'app-featured-tacos',
  standalone: true,
  imports: [CurrencyPipe],
  template: `
    <section class="featured-tacos-container">
      <h2>Featured Tacos</h2>

      <div class="taco-list">
```

```
      @for (taco of featuredTacos; track taco.name) {
        <article class="taco-card">
          <div class="taco-header">
            <h3>{{ taco.name }}</h3>
            <span class="taco-label">
              {{ taco.label }}
            </span>
          </div>

          <p class="taco-description">
            {{ taco.description }}
          </p>

          <div class="taco-footer">
            <span class="taco-price">
              {{ taco.price | currency }}
            </span>

            @if (taco.isChefsPick) {
              <div class="chefs-pick">
                Chef's Pick!
              </div>
            }
          </div>
        </article>
      }
    </div>
  </section>
  `,
})
export class FeaturedTacosComponent {
  readonly featuredTacos: FeaturedTaco[] = [
    {
      name: 'Carne Asada',
      description: 'Grilled steak with fresh cilantro and onions',
      price: 4.99,
      label: 'Popular',
      isChefsPick: false,
    },
    {
      name: 'Ghost Pepper Carnitas',
      description:
        'Slow-cooked pork with ghost peppers - not for the faint of heart',
      price: 5.49,
      label: 'Spicy',
      isChefsPick: true,
    },
    {
      name: 'Baja Fish',
      description:
        'Crispy beer-battered cod with cabbage slaw and lime crema',
      price: 6.25,
```

```
      label: 'New',
      isChefsPick: false,
    },
    {
      name: 'Al Pastor',
      description:
        'Marinated pork with pineapple and traditional spices',
      price: 4.75,
      label: 'Classic',
      isChefsPick: false,
    },
    {
      name: 'Mushroom & Poblano',
      description:
        'Roasted mushrooms and poblano peppers with queso fresco',
      price: 5.99,
      label: 'New',
      isChefsPick: false,
    },
  ];
}
```

featured-tacos.component.spec.ts

```
import { ComponentFixture, TestBed } from '@angular/core/testing';

import { FeaturedTacosComponent } from './featured-tacos.component';

describe('FeaturedTacosComponent', () => {
  let component: FeaturedTacosComponent;
  let fixture: ComponentFixture<FeaturedTacosComponent>;
  let compiled: HTMLElement;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [FeaturedTacosComponent],
    }).compileComponents();

    fixture = TestBed.createComponent(FeaturedTacosComponent);
    component = fixture.componentInstance;

    fixture.detectChanges();

    compiled = fixture.nativeElement as HTMLElement;
  });

  it('should render at least four featured tacos', () => {
    const tacoCards = compiled.querySelectorAll('.taco-card');

    expect(tacoCards.length).toBeGreaterThanOrEqual(4);
```

```
  });

  it('should render taco name, price, and featured label', () => {
    const firstTaco = component.featuredTacos[0];
    const firstCard = compiled.querySelector(
      '.taco-card',
    ) as HTMLElement;

    expect(firstCard.textContent).toContain(firstTaco.name);
    expect(firstCard.textContent).toContain('$4.99');
    expect(firstCard.textContent).toContain(firstTaco.label);
  });

  it('should display exactly one Chef\'s Pick', () => {
    const chefsPickElements =
      compiled.querySelectorAll('.chefs-pick');

    const chefsPickTacos =
      component.featuredTacos.filter(
        taco => taco.isChefsPick,
      );

    expect(chefsPickElements.length).toBe(1);
    expect(chefsPickTacos.length).toBe(1);
    expect(
      chefsPickElements[0].textContent,
    ).toContain("Chef's Pick");
  });
});
```

Developer Audit Review Notes for Adjustments to AI Produced Files

- removed unnecessary `styles: []`
- replaced legacy `*ngFor` / `*ngIf` with modern Angular `@for` / `@if`
- simplified the third unit test
- preserved the component's isolated, standalone design
- kept AI from modifying existing application files

The AI-generated solution satisfied the primary functional requirements, but I performed a developer review before integration. I made several small adjustments to better align the component with the project's Angular 22 conventions, simplify the unit tests, and remove unnecessary configuration. These changes did not expand the feature scope; they refined the generated output while preserving the controlled architecture defined in the prompt.

Run Tests Prior To Adding New Feature

jonathan@Jonathans-MacBook-Pro-M5 week7-starter % npm test -- --watch=false --browsers=FirefoxHeadless

> ng test --watch=false --browsers=FirefoxHeadless

Initial chunk files | Names | Raw size
chunk-7S2OIPMK.js | - | 2.25 MB |
polyfills.js | polyfills | 1.04 MB |
chunk-7EAYM36Y.js | - | 245.98 kB |
chunk-VRZGPI7Y.js | - | 194.02 kB |
chunk-ZQZO6AJM.js | - | 184.20 kB |
jasmine-cleanup-1.js | jasmine-cleanup-1 | 67.12 kB |
chunk-BXQY7UK7.js | - | 40.46 kB |
styles.css | styles | 23.82 kB |
chunk-HMD54CUK.js | - | 20.97 kB |
chunk-V2CRO6DE.js | - | 10.79 kB |
spec-app-order-order.component.spec.js | spec-app-order-order.component.spec | 10.26 kB |
spec-app-feedback-feedback.component.spec.js | spec-app-feedback-feedback.component.spec | 9.27 kB |
spec-app-order-summary-order-summary.component.spec.js | spec-app-order-summary-order-summary.component.spec | 9.15 kB |
chunk-4NK4ZTQM.js | - | 8.38 kB |
spec-app-daily-specials-daily-specials.component.spec.js | spec-app-daily-specials-daily-specials.component.spec | 8.28 kB |
chunk-3OQY4ED3.js | - | 7.35 kB |
spec-app-signin-signin.component.spec.js | spec-app-signin-signin.component.spec | 6.36 kB |
spec-app-app.component.spec.js | spec-app-app.component.spec | 5.85 kB |
spec-app-home-home.component.spec.js | spec-app-home-home.component.spec | 5.41 kB |
test_main.js | test_main | 4.55 kB |
chunk-URFXOZKS.js | - | 3.54 kB |
chunk-5XADRTYU.js | - | 3.35 kB |
chunk-TRXK2FCV.js | - | 2.26 kB |
spec-app-auth.service.spec.js | spec-app-auth.service.spec | 1.64 kB |
spec-app-menu-menu.component.spec.js | spec-app-menu-menu.component.spec | 1.28 kB |
spec-app-auth.guard.spec.js | spec-app-auth.guard.spec | 1.11 kB |
jasmine-cleanup-0.js | jasmine-cleanup-0 | 397 bytes |

| Initial total | 4.16 MB

Application bundle generation complete. [0.605 seconds] - 2026-10-12T04:18:10.131Z

12 10 2026 00:18:10.219:INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
12 10 2026 00:18:10.219:INFO [launcher]: Launching browsers FirefoxHeadless with concurrency unlimited

12 10 2026 00:18:10.222:INFO [launcher]: Starting browser FirefoxHeadless
12 10 2026 00:18:10.986:INFO [Firefox 155.0 (Mac OS 10.15)]: Connected on socket g0wzSPaWSmBx0FSmAAAB with id 55876454
Firefox 155.0 (Mac OS 10.15): Executed 32 of 32 SUCCESS (0.15 secs / 0.123 secs)
TOTAL: 32 SUCCESS

Add New Feature

We first need to add the new route to `src/app/app.routes.ts`:

```
{
  path: 'featured-tacos',
  loadComponent: () =>
    import('./featured-tacos/featured-tacos.component')
      .then(m => m.FeaturedTacosComponent),
},
```

Then we run the tests again:

jonathan@Jonathans-MacBook-Pro-M5 week7-starter % vi src/app/app.routes.ts
jonathan@Jonathans-MacBook-Pro-M5 week7-starter % npm test -- --watch=false --browsers=FirefoxHeadless

> virtual-taco-stand@0.0.0 test
> ng test --watch=false --browsers=FirefoxHeadless

Initial chunk files | Names | Raw size
chunk-7S2OIPMK.js | - | 2.25 MB |
polyfills.js | polyfills | 1.04 MB |
chunk-7EAYM36Y.js | - | 245.98 kB |
chunk-VRZGPI7Y.js | - | 194.02 kB |
chunk-ZQZO6AJM.js | - | 184.20 kB |
jasmine-cleanup-1.js | jasmine-cleanup-1 | 67.12 kB |
chunk-BXQY7UK7.js | - | 40.46 kB |
styles.css | styles | 23.82 kB |
chunk-HMD54CUK.js | - | 20.97 kB |
chunk-V2CRO6DE.js | - | 10.79 kB |
spec-app-order-order.component.spec.js | spec-app-order-order.component.spec | 10.26 kB |
spec-app-feedback-feedback.component.spec.js | spec-app-feedback-feedback.component.spec | 9.27 kB |
spec-app-order-summary-order-summary.component.spec.js | spec-app-order-summary-order-summary.component.spec | 9.15 kB |
chunk-4NK4ZTQM.js | - | 8.38 kB |
spec-app-daily-specials-daily-specials.component.spec.js | spec-app-daily-specials-daily-specials.component.spec | 8.28 kB |
chunk-3OQY4ED3.js | - | 7.35 kB |

spec-app-signin-signin.component.spec.js | spec-app-signin-signin.component.spec | 6.36 kB |
spec-app-app.component.spec.js | spec-app-app.component.spec | 5.85 kB |
spec-app-home-home.component.spec.js | spec-app-home-home.component.spec | 5.41 kB |
spec-app-featured-tacos-featured-tacos.component.spec.js | spec-app-featured-tacos-featured-tacos.component.spec | 4.78 kB |
test_main.js | test_main | 4.55 kB |
chunk-URFXOZKS.js | - | 3.54 kB |
chunk-5XADRTYU.js | - | 3.35 kB |
chunk-TRXK2FCV.js | - | 2.26 kB |
spec-app-auth.service.spec.js | spec-app-auth.service.spec | 1.64 kB |
spec-app-menu-menu.component.spec.js | spec-app-menu-menu.component.spec | 1.28 kB |
spec-app-auth.guard.spec.js | spec-app-auth.guard.spec | 1.11 kB |
jasmine-cleanup-0.js | jasmine-cleanup-0 | 397 bytes |

| Initial total | 4.17 MB

Application bundle generation complete. [0.519 seconds] - 2026-10-12T04:23:37.362Z

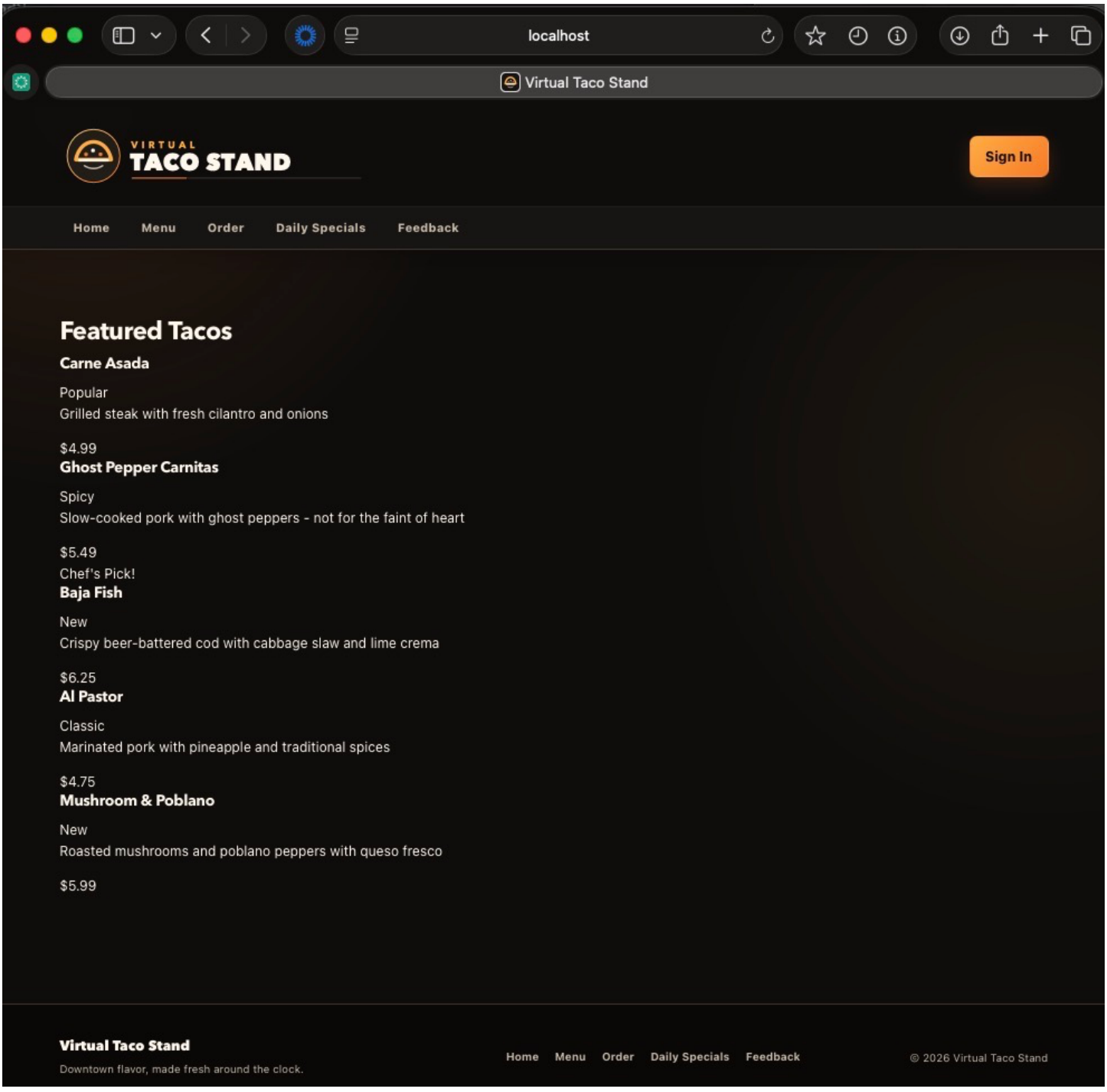
12 10 2026 00:23:37.438:INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
12 10 2026 00:23:37.439:INFO [launcher]: Launching browsers FirefoxHeadless with concurrency unlimited
12 10 2026 00:23:37.441:INFO [launcher]: Starting browser FirefoxHeadless
12 10 2026 00:23:38.137:INFO [Firefox 155.0 (Mac OS 10.15)]: Connected on socket tuCHqRwKux4Bn72XAAAB with id 77753854
Firefox 155.0 (Mac OS 10.15): Executed 35 of 35 SUCCESS (0.15 secs / 0.126 secs)
TOTAL: 35 SUCCESS

Everything looks good!

Start the site locally to inspect the running feature:

```
npm start
```

Visit the site: http://localhost:4200/featured-tacos



Validation and Explanation

For this assignment, I added a new Featured Taco List component to the Virtual Taco Stand application. The component displays five featured tacos with a name, description, price, and descriptive label for each item. One taco is conditionally identified as the Chef's Pick using Angular template control flow. The component was implemented as a standalone Angular component and keeps its data and display behavior self-contained. I integrated the feature through a new `featured-tacos` route without modifying any existing application components or order-processing logic.

Before adding the feature, I ran the complete unit test suite and confirmed that all 32 existing tests passed. I then added the new component and its three unit tests and reran the full test suite. The first new test verifies that at least four featured tacos are rendered. The second verifies that the displayed taco information includes the expected name, price, and featured label. The third verifies that exactly one taco is displayed as the Chef's Pick. After integration, all 35 tests passed, confirming that the three new tests succeeded and that no existing tests regressed. I also started the application locally and manually reviewed the `/featured-tacos` route to confirm that the taco information, prices, labels, and Chef's Pick were displayed correctly.

Before accepting the AI-generated output, I reviewed its Angular structure, imports, template syntax, test design, and adherence to the scope defined in my prompt. I made limited revisions, including replacing `*ngFor` and `*ngIf` with modern Angular `@for` and `@if` syntax, removing unnecessary configuration, and simplifying the unit tests without expanding the feature scope. This approach differed from Week 7 because the AI was given explicit architectural boundaries, its output was reviewed before integration, and the application was validated against a known passing baseline before the change was accepted.